

Graph Neural Networks, Homomorphisms and Logic

Carsten Lutz
Universität Leipzig

Joint work with

Balder ten Cate, Maurice Funk, Benny Kimmelfeld, Moritz Schönherr, Arie Soetemann



Graph Learning

Graph Learning is important subarea of machine learning:

- many real world data represented as graphs
social networks, molecule graphs, recommender systems, knowledge graphs
- standard ML models such as CNNs and RNNs require serialization / ordering
obscures graph structure and results in classifiers that are not isomorphism-invariant

Many ML approaches to graph learning have been explored, e.g. kernel methods:

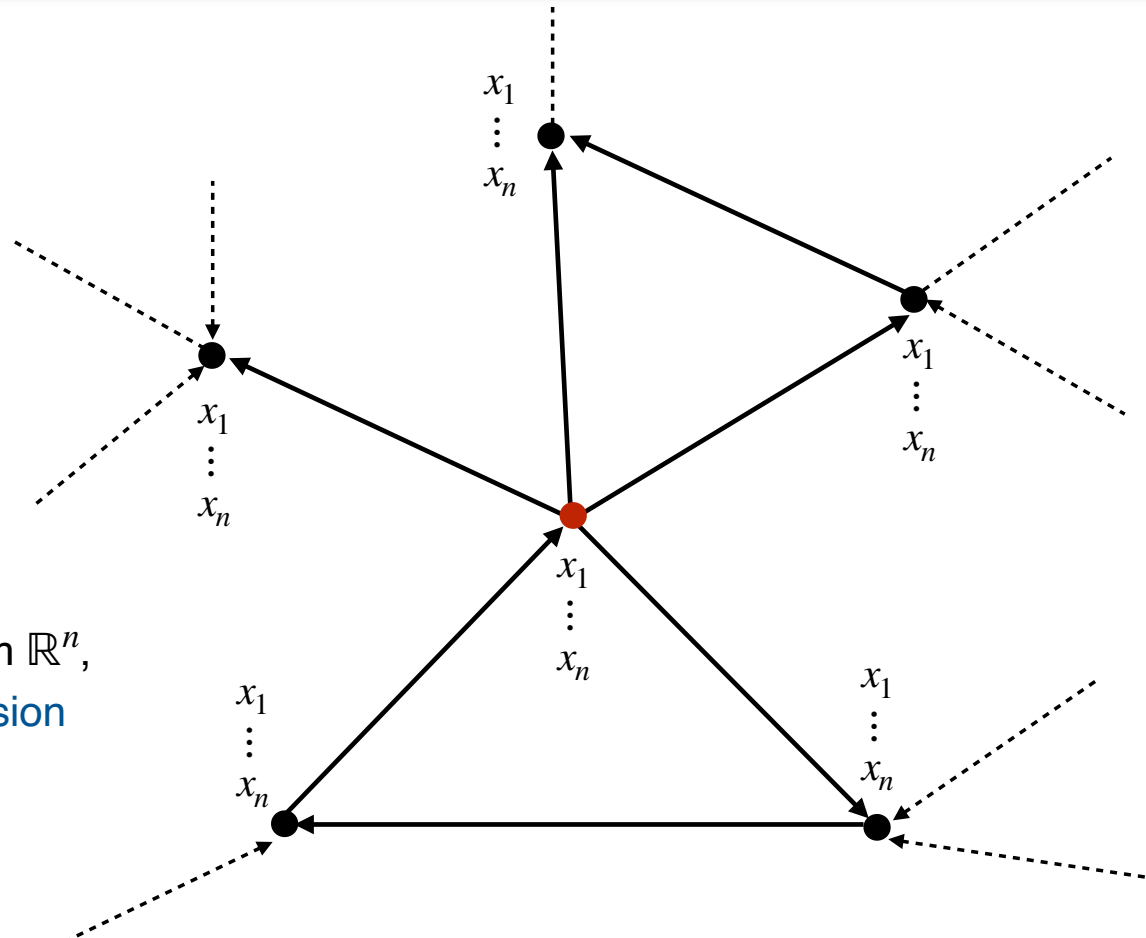
extract a lot of features (number of certain paths, subgraphs, WL colors, etc)
then use linear classifier such as SVM

Since a few years, Graph neural networks (GNNs) [Scarselli et al. 2005] dominate the area

Also known by other names: GCN, GraphSAGE, GIN, etc

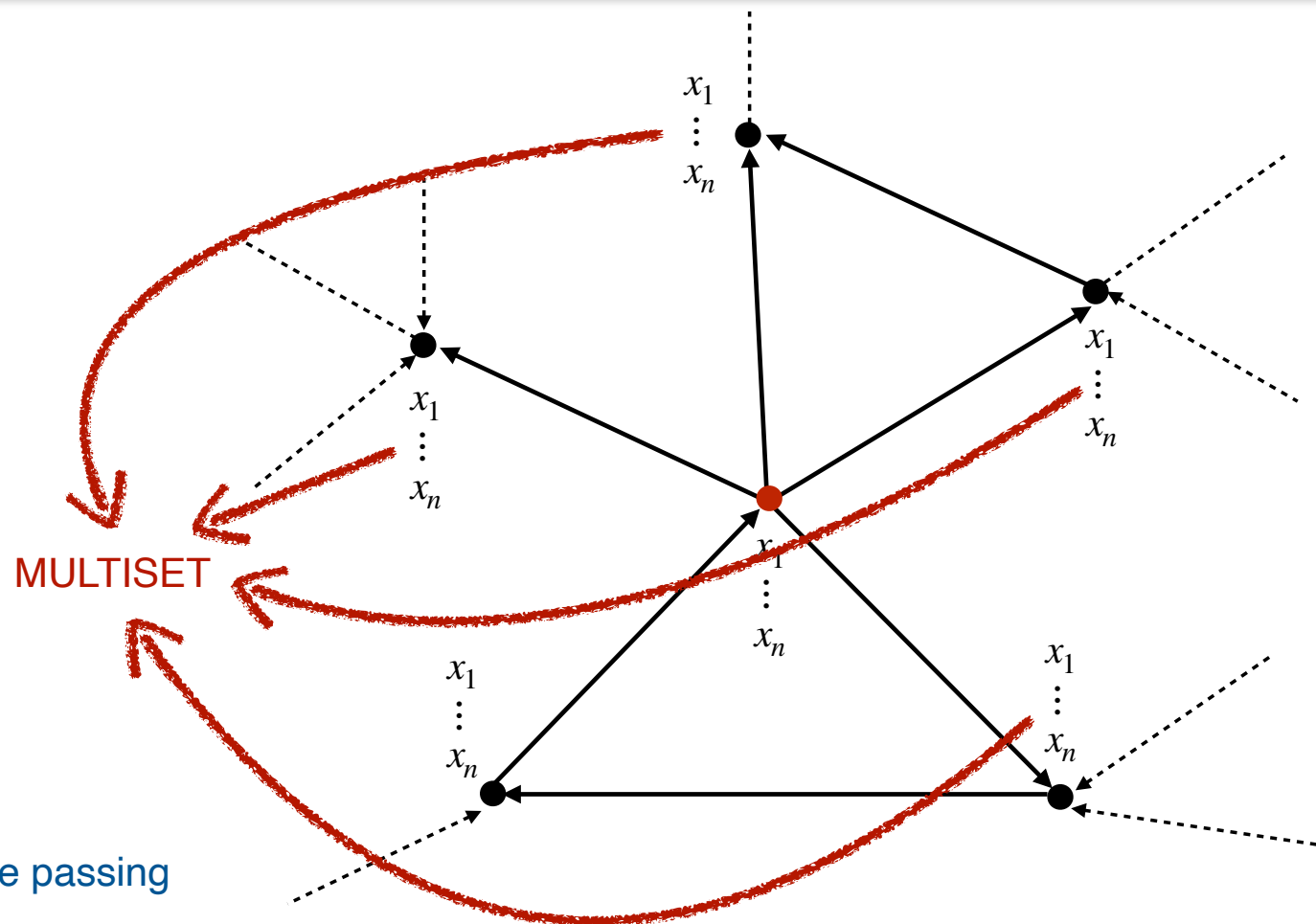


Aggregate-Combine GNNs

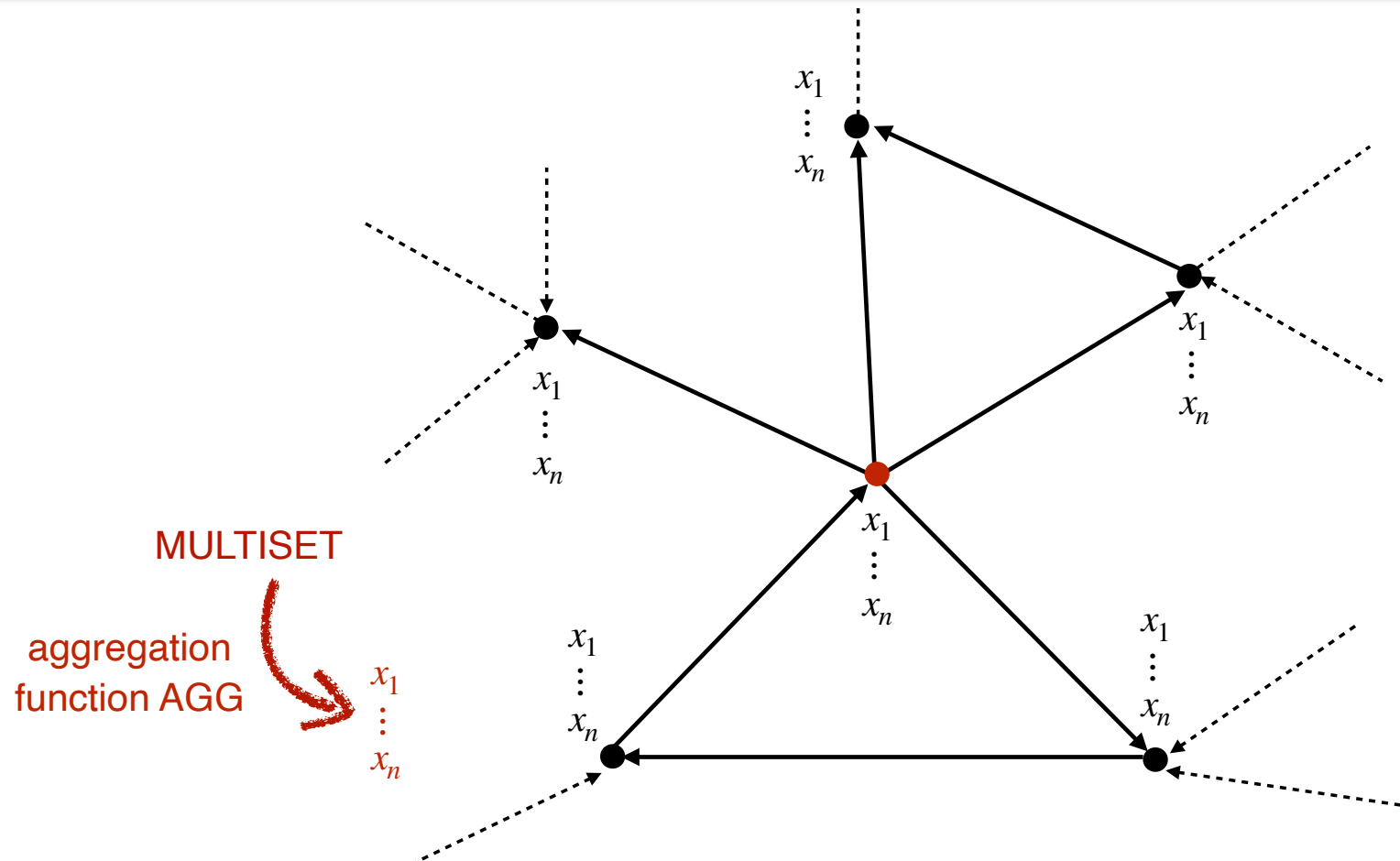


Embedding vectors from $\mathbb{R}^n$,
 n internal GNN dimension

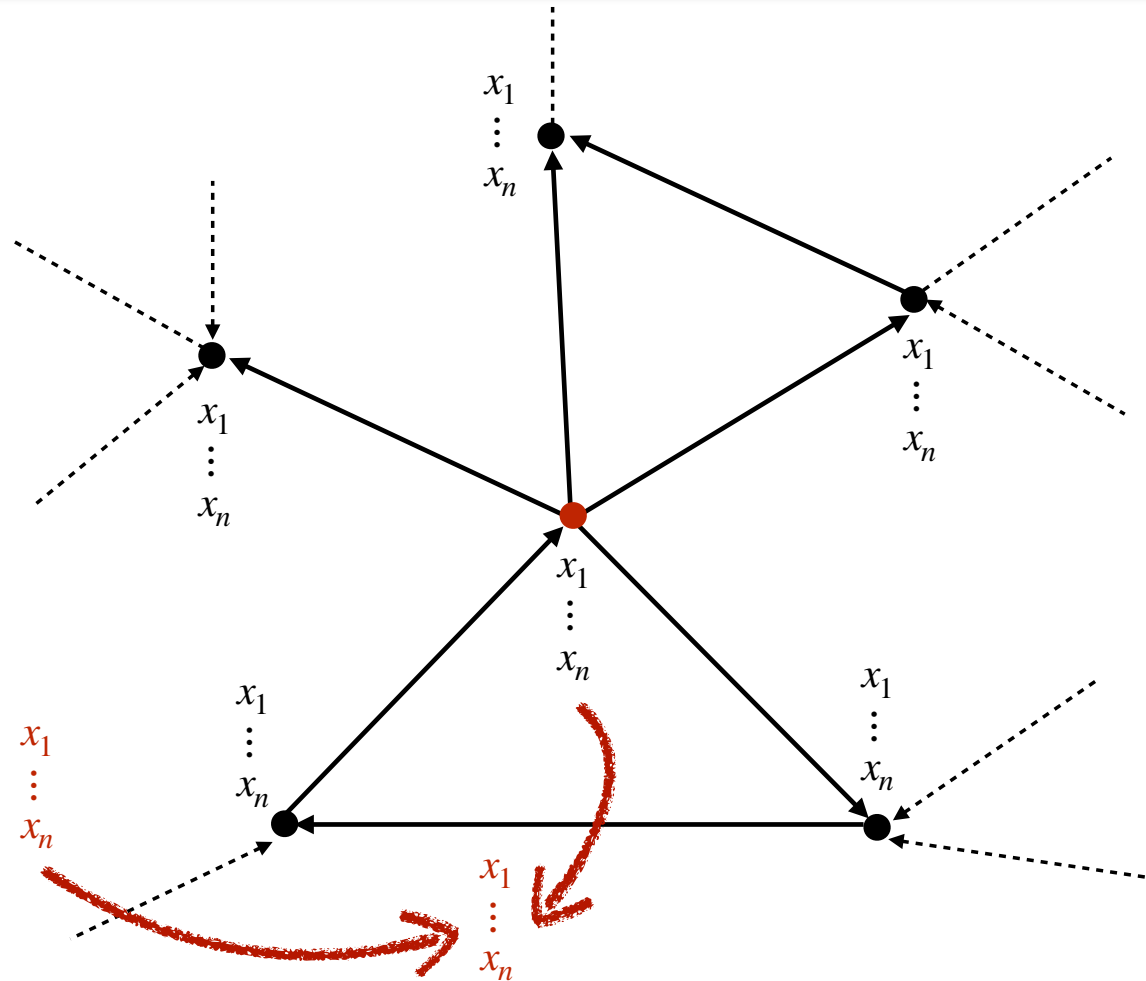
Aggregate-Combine GNNs



Aggregate-Combine GNNs

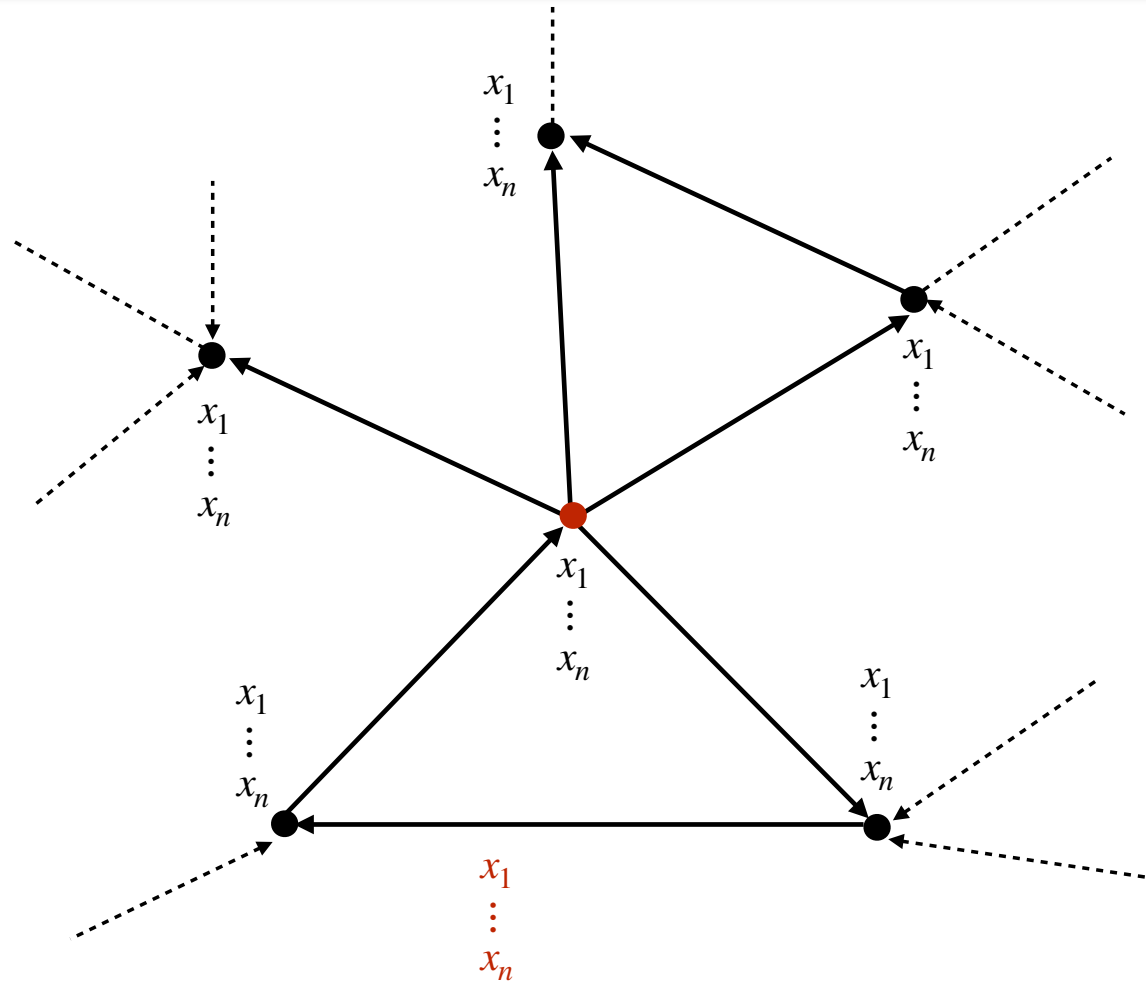


Aggregate-Combine GNNs



combination
function COM

Aggregate-Combine GNNs



GNN transforms
vertex embeddings
into vertex embeddings

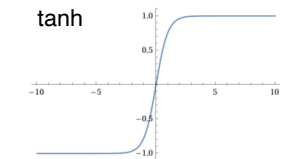
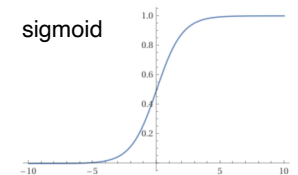
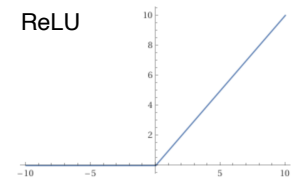
Aggregate-Combine GNNs

A basic case:

- constant number of iterations / layers
- classification function CLS maps final embedding to vertex classification (graph classification, link prediction, etc also possible)

Architectural choices:

- aggregation function AGG: sum, mean, max (all applied component-wise)
- combination function COM represented by feedforward neural network (FFN)
 - non-linear activation function ACT at every neuron, e.g. ReLU, sigmoid
 - weights of neural network determined when GNN is learned (gradient descent)
- classification function CLS: simple threshold



GNNs versus WL

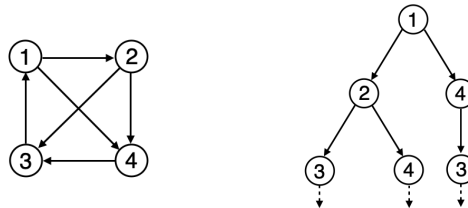
A fundamental result on the expressive power of GNNs:

Theorem [Morris et al. 2019/Xu et al. 2019]

If two vertices of a graph cannot be distinguished by the **1-dimensional Weisfeiler-Leman test**, then they cannot be distinguished by a GNN.

Therefore...

- GNNs cannot distinguish between graph and its **infinite unraveling into a tree**



- GNNs cannot detect **basic subgraph patterns** such as triangles, 4-cycles, 4-cliques, etc

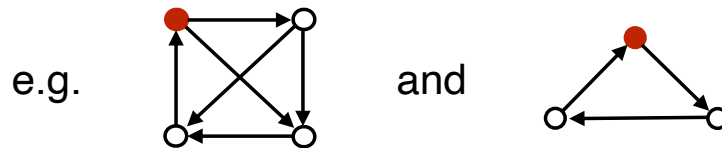
Various proposals to make GNNs more expressive:

high-order GNNs, subgraph GNNs, GNNs with local graph parameters, hierarchical ego-GNNs, etc

Homomorphism Counts

GNNs with homomorphism count features [Bouritsas et al 2020, Barcelo et al 2021]:

- as part of model design, choose a set of **pointed graph patterns**



- for every pattern, add **homomorphism count** to every vertex **as a feature**
- afterwards **run GNN** as before

Pro: able to detect non-trivial subgraph patterns, show **improved performance in practice**

Con: graph structure and learning **somewhat detached** (c.f. kernel methods)

Deep Homomorphism Networks

Deep Homomorphisms Networks (DHNs) [MaeharaNT 2024]:

- integrate **pattern graph homomorphism count** into GNN iteration
- homomorphism now into **vertex-embedded** graph
- requires **two nested aggregations**:
 1. inside a single homomorphism, **aggregate over all vertices in the image**
 2. **aggregate over all homomorphisms** (sum, mean, max)
- vertices in pattern graphs may “play different roles”
 - ~ associate **transformation function** with each pattern graph vertex,
apply before aggregation

We want to **understand the expressive power of DHNs, via logic**



Deep Homomorphism Networks

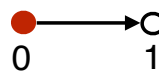
Homomorphism query: triple $(F^\bullet, \mu, \text{agg})$ where

- $F^\bullet$ is a **pointed graph pattern**
- μ labels each vertex u in $F^\bullet$ with a **transformation function** $\mu_u : \mathbb{R}^d \rightarrow \mathbb{R}^{d'}$
- agg is an **aggregation function** such as component-wise sum, mean, max

Evaluating a hom query at a vertex v in an embedded graph (G, λ) :

- every homomorphism h from $F^\bullet$ to G with $h(\bullet) = v$
gives **embedding of the vertices of $F^\bullet$** : $\text{emb}_h(u) = \mu_u(\lambda(h(u)))$ for all $u \in V^F$
- **result of hom query at v** : $\text{agg}\{\{\prod_{u \in V^F} \text{emb}_h(u) \mid h \in \text{Hom}(F^\bullet, D^v)\}\}$
could be chosen otherwise

Results of **all chosen hom queries** fed into **combination function** $\leadsto$ new vertex embedding

Gives exactly GNNs for patterns  and 

global readout:  

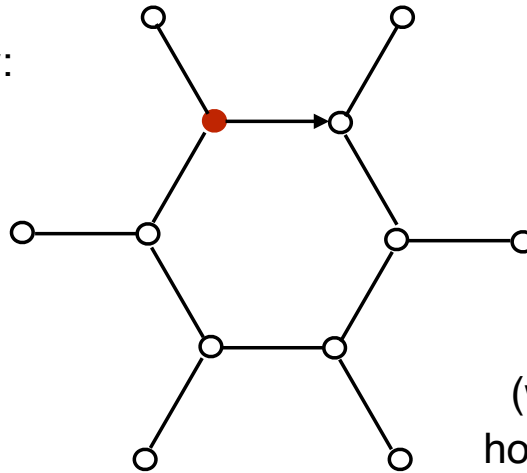
A First Result

Indeed, we gain something:

Theorem

Sum-DHNs are more expressive than Sum-GNNs with homomorphism count features

Separating property:



where every “satellite vertex”
has exactly one neighbor

(we make use of a result by Chen et al. [2025] that
homomorphism counts cannot detect isolate vertices)

Baseline: Max Aggregation

Max aggregation **disables counting** and often admits **clean characterizations of expressive power**

Theorem

Max-DHN = UNFO = **HML**

UNFO = **Unary negation fragment of first-order logic**:

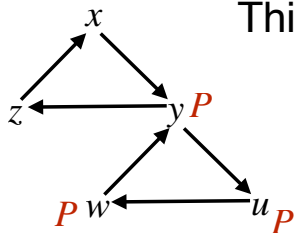
$$\varphi(\bar{x}), \psi(\bar{x}) ::= R(\bar{x}) \mid \varphi(\bar{x}) \wedge \psi(\bar{x}) \mid \varphi(\bar{x}) \vee \psi(\bar{x}) \mid \exists y \varphi(\bar{x}, y) \mid \neg \varphi(\bar{x})$$

Equivalent to the following **Homomorphism Modal Logic (HML)**:

$$\varphi(x), \psi(x) ::= \neg \varphi(x) \mid \varphi(x) \wedge \psi(x) \mid \exists \bar{y} \vartheta(x, \bar{y})$$

conjunction of atoms
(i.e., conjunctive query)

Think of this as “the modal logic of conjunctive queries”, e.g.



$$\varphi(x) = \exists y, z \left(E(x, y) \wedge E(y, z) \wedge E(z, x) \wedge \right. \\ \left. \forall u, w \left((E(y, u) \wedge E(u, w) \wedge E(w, y)) \rightarrow (P(y) \wedge P(u) \wedge P(w)) \right) \right)$$

Baseline: Max Aggregation

Max aggregation **disables counting** and often admits **clean characterizations of expressive power**

Theorem

Max-DHN = UNFO = HML

- “ $\subseteq$ ”:
- **only finitely many embedding vectors** occur at each layer, across all graphs
 - for each vector, HML can define **all substructures that produce them**
(pattern graph $\rightsquigarrow$ conjunctive query)
 - at final layer, “big disjunction” encodes classification function

very general: any
COM, TRS, CLS
function

“ $\supseteq$ ”: Let φ be HML formula; idea to build equivalent Max-DHN:

Embedding vectors: one position for every subformula ψ of φ
value 1 if ψ true at vertex and 0 if ψ false at vertex

ReLU ACT,
threshold CLS

One DHN layer per subformula, from simpler to more complex formulas



Sum Aggregation

Sum aggregation adds **counting capabilities**

UNFOC: $\varphi(\bar{x}), \psi(\bar{x}) ::= R(\bar{x}) \mid \varphi(\bar{x}) \wedge \psi(\bar{x}) \mid \varphi(\bar{x}) \vee \psi(\bar{x}) \mid \exists^{\geq n} y \varphi(\bar{x}, y) \mid \neg \varphi(x_i)$

GHML: $\varphi(x), \psi(x) ::= \neg \varphi(x) \mid \varphi(\bar{x}) \wedge \psi(\bar{x}) \mid \exists^{\geq n_1} y_1 \dots \exists^{\geq n_m} y_m \vartheta(x, y_1, \dots, y_m)$

Theorem

1. Sum-DHN $\supseteq$ GHML
2. connected Sum-DHN $\subseteq$ connected GHML, on graphs of bounded degree

@bounded degree: this cannot be dropped, even for Sum-GNNs:

Sum-GNNs can express “**more red successors than green successors**”, GHML cannot

@connected: same, with “graph vertices” in place of “successors”

but we can say a bit more!



Sum Aggregation

It is well-known that:

Theorem [Barcelo et al 2019]

$\text{Sum-GNN} \cap \text{FO} = \text{graded modal logic}$

By contrast:

Theorem

$\text{connected Sum-DHN} \cap \text{FO} \not\supseteq \text{connected GHML}$

Separating property:

local transitivity, i.e. $\varphi(x) = \forall y \forall z ((E(x, y) \wedge E(y, z)) \rightarrow E(x, z))$

In a DHN: use  and , then subtract

Mean Aggregation

Arithmetic mean is popular aggregation function in practice because it admits sampling

Known from GNNs [Xu et al. 2019]:

- “absolute counting” (e.g. at least 3 red successors) no longer possible
- “relative counting” (e.g. at least 1/2 of the successors red) still possible

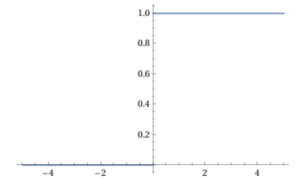
Mean Aggregation

Ratio HML (RHML):

$$\varphi(x) ::= \neg\varphi(x) \mid \varphi(x) \wedge \varphi(x) \mid \frac{\exists_{\sim r} \bar{y} (\mu(x, \bar{y}), \nu(x, \bar{y}))}{\vartheta(x)} \quad \sim \in \{\geq, >\} \quad r \in [0,1]$$

$G \models \vartheta(v)$ with $|\bar{y}| = n$ if

$$\frac{|\{\bar{w} \in (V^G)^n \mid G \models \mu(v, \bar{w}) \wedge \nu(v, \bar{w})\}|}{|\{\bar{w} \in (V^G)^n \mid G \models \nu(v, \bar{w})\}|} \sim r$$



e.g. at least 50% of all triangles rooted in this vertex have reflexive loops at all three points

Theorem

1. Mean-DHN $\supseteq$ RHML
2. connected Mean-DHN $\subseteq$ connected RHML, on graphs of bounded degree

Point 1 **fails** for ReLU, uses (discontinuous) **step function**; see [SchönherrL__2026]



Improved Lower Bound

Let's get back to:

Theorem

Sum-DHN $\not\supseteq$ GHML, separating FO-property local transitivity

Can we find a **logical lower bound** that **encompasses local transitivity**?

Maybe even matches $\text{Sum-DHN} \cap \text{FO}$?

UQAFO (unary quantifier-alternation fragment of FO):

- atomic (non-unary) negation, unrestricted duals, atoms $x_1 = x_2$, $x_1 \neq x_2$ permitted
- **quantifier alternation** is restricted to unary subformulas

Examples:

$\exists x \exists y (\forall z R(x, z) \wedge \forall z S(y, z))$ 

$\exists x \exists y \forall z (E(x, z) \wedge E(z, y))$ 

$\forall x \forall y (E(x, z) \wedge E(z, y) \rightarrow E(x, y))$  (local transitivity)

Improved Lower Bound

Let's get back to:

Theorem

Sum-DHN $\supsetneq$ GHML, separating FO-property local transitivity

Can we find a **logical lower bound** that **encompasses local transitivity**?

Maybe even matches $\text{Sum-DHN} \cap \text{FO}$?

UQAFO (unary quantifier-alternation fragment of FO):

- atomic (non-unary) negation, unrestricted duals, atoms $x_1 = x_2$, $x_1 \neq x_2$ permitted
- **quantifier alternation** is restricted to unary subformulas

UQAFO indeed provides a lower bound:

Theorem

$\text{UNFOC} \subsetneq \text{UQAFO} \subseteq \text{Sum-DHN}$

From Homomorphism to ...

What is a corresponding version of HML?



From Homomorphism to Embeddings

What is a corresponding version of HML?

EML (embedding modal logic):

= HML, but **homomorphisms replaced with embeddings** (injective and also preserve non-facts)

Lemma

UQAFO = EML

Can also put **embeddings into DHNs** $\leadsto$ **DENs**

Lovász homomorphism counting theorem yields:

Theorem

Sum-DHN = Sum-DEN

Is this also true for Max aggregation? No! And UQAFO is not just some random logic!

Theorem

Max-DEN = UQAFO = EML $\supsetneq$ Max-DHN

From Homomorphism to Embeddings

...maybe even matches $\text{Sum-DHN} \cap \text{FO}$?

Unfortunately not:

Theorem

$\text{UQAFO} \subsetneq \text{Sum-DHN} \cap \text{FO}$

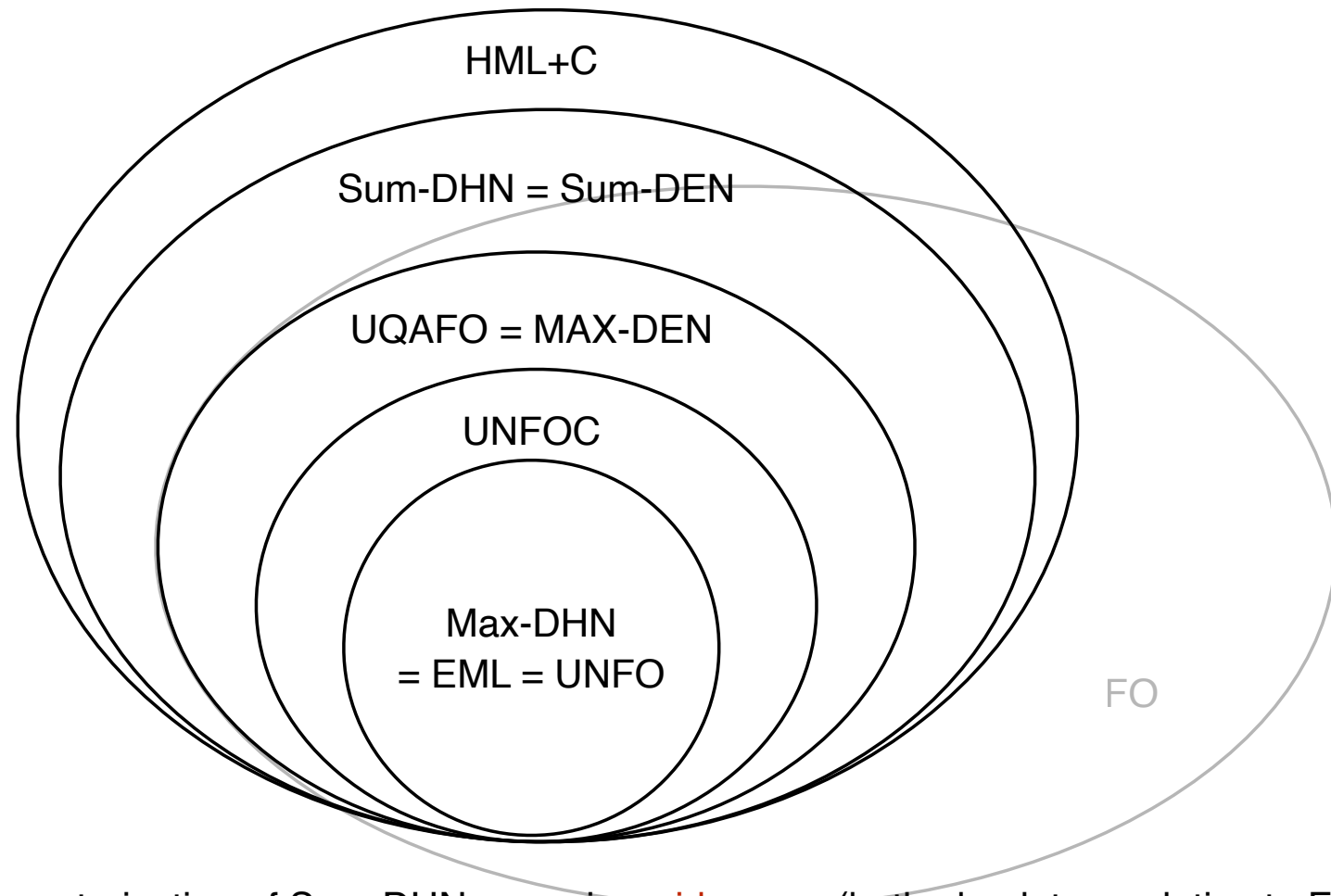
Separating property:

Descendants of v form linear order $v_0, \dots, v_n$ such that v_i colored $P_{i \bmod 3}$ for $0 \leq i \leq n$

We also give an upper bound: **HML+C**

i.e., HML with very expressive counting in the style of FO+C used in descriptive complexity

Overview



Exact logical characterization of Sum-DHNs remains **wide open** (both absolute + relative to FO)

Static Analysis

We can use the DHN-logic connection to analyse some **fundamental static analysis problems**

Emptiness: given a DHN, decide whether it defines the **empty vertex classifier**

Subsumption: given DHNs $\mathcal{G}_1, \mathcal{G}_2$, decide whether the defined properties P_1, P_2 satisfy $P_1 \subseteq P_2$

Theorem

1. For max-DHNs, emptiness and subsumption are decidable (2Exp-hard)
2. For connected sum-DHNs, emptiness and subsumption are decidable on databases of bounded degree (coNExp-complete)
3. For sum-DHNs, emptiness and subsumption are undecidable

This already holds

- (a) connected sum-DHNs
- (b) sum-DHNs on graphs of bounded degree.

DHNs and Databases

DHNs are attractive for **database applications**:

- the generalization of GNNs to **higher arity relations** is **formally awkward**
for DHNs, by contrast, it is **very smooth** (almost nothing changes)
- homomorphism queries = **conjunctive queries**,
the workhorse of database theory + systems

Can we go further?

In-database implementation of DHNs? Integration of querying + neural computation?

Neuro-Relational Programs

Embedded database:

- every relation has an arity and an **embedding dimension d**
- **embedded fact** has form $R(\bar{a})\langle\bar{e}\rangle$ with $R(\bar{a})$ a fact and $\bar{e} \in \mathbb{R}^d$
cannot have $R(\bar{a})\langle\bar{e}_1\rangle, R(\bar{a})\langle\bar{e}_2\rangle \in D$ with $e_1 \neq e_2$

c.f. GNNs/DHNs:
vertex embeddings!

Neuro-Relational Program (NRP) transforms embedded database into embedded relation

Three types of rules:

- **conjunction rules**
$$R(\bar{x})\langle\text{agg}\rangle \Leftarrow_{\text{com}} R_1(\bar{x}_1) \wedge \dots \wedge R_\ell(\bar{x}_\ell)$$
- **disjunction rules**
$$R(\bar{x})\langle\text{agg}\rangle \Leftarrow R_1(\bar{x}) \vee \dots \vee R_\ell(\bar{x})$$
- **transformation rules**
$$R(\bar{x})\langle\text{tr}\rangle \Leftarrow R_0(\bar{x})$$

Must be **non-recursive** and have **unique head relations**, one of them the **goal relation**

Neuro-Relational Programs

Embedded database:

- every relation has an arity and an **embedding dimension d**
- embedded fact** has form $R(\bar{a})\langle\bar{e}\rangle$ with $R(\bar{a})$ a fact and $\bar{e} \in \mathbb{R}^d$
cannot have $R(\bar{a})\langle\bar{e}_1\rangle, R(\bar{a})\langle\bar{e}_2\rangle \in D$ with $e_1 \neq e_2$

c.f. GNNs/DHNs:
vertex embeddings!

Neuro-Relational Program (NRP) transforms embedded database into embedded relation

Three types of rules:

- conjunction rules** $R(\bar{x})\langle\text{agg}\rangle \leftarrow_{\text{com}} R_1(\bar{x}_1) \wedge \cdots \wedge R_\ell(\bar{x}_\ell)$

works like a **homomorphism query without transformation functions**

(com replaces Π , agg is aggregation across homomorphisms)

Neuro-Relational Programs

Embedded database:

- every relation has an arity and an **embedding dimension d**
- **embedded fact** has form $R(\bar{a})\langle\bar{e}\rangle$ with $R(\bar{a})$ a fact and $\bar{e} \in \mathbb{R}^d$
cannot have $R(\bar{a})\langle\bar{e}_1\rangle, R(\bar{a})\langle\bar{e}_2\rangle \in D$ with $e_1 \neq e_2$

c.f. GNNs/DHNs:
vertex embeddings!

Neuro-Relational Program (NRP) transforms embedded database into embedded relation

Three types of rules:

- **conjunction rules** $R(\bar{x})\langle\text{agg}\rangle \Leftarrow_{\text{com}} R_1(\bar{x}_1) \wedge \dots \wedge R_\ell(\bar{x}_\ell)$
- **disjunction rules** $R(\bar{x})\langle\text{agg}\rangle \Leftarrow R_1(\bar{x}) \vee \dots \vee R_\ell(\bar{x})$

rule produces **fact** $R(\bar{a})\langle\bar{e}\rangle$ if $R_i(\bar{a}) \in D$ for some i , with

$$\bar{e} = \text{agg}\{\{\bar{e} \mid R_i(\bar{a})\langle\bar{e}\rangle \in D \text{ with } 1 \leq i \leq \ell\}\}$$

Neuro-Relational Programs

Embedded database:

- every relation has an arity and an **embedding dimension d**
- **embedded fact** has form $R(\bar{a})\langle\bar{e}\rangle$ with $R(\bar{a})$ a fact and $\bar{e} \in \mathbb{R}^d$
cannot have $R(\bar{a})\langle\bar{e}_1\rangle, R(\bar{a})\langle\bar{e}_2\rangle \in D$ with $e_1 \neq e_2$

c.f. GNNs/DHNs:
vertex embeddings!

Neuro-Relational Program (NRP) transforms embedded database into embedded relation

Three types of rules:

- **conjunction rules**

$$R(\bar{x})\langle\text{agg}\rangle \Leftarrow_{\text{com}} R_1(\bar{x}_1) \wedge \cdots \wedge R_\ell(\bar{x}_\ell)$$

- **disjunction rules**

$$R(\bar{x})\langle\text{agg}\rangle \Leftarrow R_1(\bar{x}) \vee \cdots \vee R_\ell(\bar{x})$$

- **transformation rules**

$$R(\bar{x})\langle\text{tr}\rangle \Leftarrow R_0(\bar{x})$$

rule produces **fact** $R(\bar{a})\langle\text{tr}(\bar{e})\rangle$ for every $R_0(\bar{a})\langle\bar{e}\rangle \in D$

combination and transformation
functions are represented by FNNs,
trainable

Example

Database-to-graph translation and application of GNN can both be written as NRP:

$CNode(cid)\langle SUM \rangle$	$\Leftarrow$	$Customer(cid, s) \wedge SegEmb(s)$	Import features as initial embeddings from database
$PNode(pid)\langle SUM \rangle$	$\Leftarrow$	$Product(pid, k) \wedge CatEmb(k)$	
$ONode(oid)\langle SUM \rangle$	$\Leftarrow_{\oplus}$	$Order(oid, cid, pid, t, r) \wedge CountryEmb(t) \wedge ChanEmb(r)$	
$Node(x)\langle SUM \rangle$	$\Leftarrow$	$CNode(x) \vee PNode(x) \vee ONode(x)$	
$PlacedBy(oid, cid)$	$\Leftarrow$	$Order(oid, cid, pid, t, r)$	Construct graph from relations
$Contains(oid, pid)$	$\Leftarrow$	$Order(oid, cid, pid, t, r)$	
$Edge(u, v)$	$\Leftarrow$	$PlacedBy(u, v) \vee Contains(u, v)$	
$Aggr(u)\langle SUM \rangle$	$\Leftarrow$	$Edge(u, v) \wedge Node(v)$	Run one layer of GNN
$AggrZeroes(u)\langle \mu_0 \rangle$	$\Leftarrow$	$Node(u)$	
$AggrOrZeroes(u)\langle SUM \rangle$	$\Leftarrow$	$Aggr(u) \vee AggrZeroes(u)$	
$Concat(u)$	$\Leftarrow_{\oplus}$	$Node(u) \wedge AggrOrZeroes(u)$	
$Node'(u)\langle \rho \rangle$	$\Leftarrow$	$Concat(u)$	

This extends to DHNs!

Neuro-Relational Programs

We concentrate on

- **sum** as aggregation function (component-wise)
- **product** as combination function

combination functions **sum** and **concatenation** can be simulated by product

In the following, a **query** transforms embedded DBs to embedded relations

A **Boolean query** may be defined via an additional **acceptance policy**



Everything is Connected

What is the expressive power of NRPs? Result overview:

- zero-ary NRPs = non-adaptive query algorithms
 - monadic NRPs = Sum-DHNs
 - frontier-guarded NRPs with unique row-ids $\subseteq$ Sum-DHNs
 - unrestricted NRPs = TC^0 over ordered classical databases
- } refers to **arity of head relations!**

Also, NRMs may be viewed as a special case of the more general [LARA framework](#) [BarceloEtAl 2022]



Zero-Ary NRPs

Non-adaptive query algorithm is tuple $A = (F_1, \dots, F_k, X)$ with

- $F_1, \dots, F_k$ finite databases and
- $X \subseteq \mathbb{N}^k$

A accepts database D if $(|\text{Hom}(F_1, D)|, \dots, |\text{Hom}(F_k, D)|) \in X$

Generalizes Graph Motif Parameters where X defined by linear combination

Zero-Ary NRPs

Non-adaptive query algorithm is tuple $A = (F_1, \dots, F_k, X)$ with

- $F_1, \dots, F_k$ finite databases and
- $X \subseteq \mathbb{N}^k$

A accepts database D if $(|\text{Hom}(F_1, D)|, \dots, |\text{Hom}(F_k, D)|) \in X$

Theorem

non-adaptive query algorithms = Zero-ary NRPs with unrestricted transformation functions
(for Boolean queries over classical databases)

We thus get from existing results [ChenEtAl 2025]:

Corollary

1. alternation-free FO $\subseteq$ zero-ary NRPs
2. $\exists x \forall y \neg R(x, y)$ cannot be computed by a zero-ary NRP
3. “even number of triangles” can be computed by zero-ary NRP

Monadic NRPs

In a **unary-embedded** database, only unary relations have non-zero embedding dimension

Unary-embedded DBs (for NRPs) **intertranslatable** with vertex-embedded DBs (for DHNs)

Theorem

Monadic NRPs = Sum-DHNs
(for unary queries over unary-embedded databases)

The results from the first part thus give us:

Corollary

1. $\text{UQAFO} \subseteq \text{monadic NRPs}$
2. $\text{monadic NRPs} \cap \text{FO} \not\subseteq \text{UQAFO}$

Frontier-Guarded NRPs

Frontier-guarded NRPs: all variables in rule head must co-occur in atom in rule body

Theorem

Frontier-guarded NRPs are strictly more expressive than monadic NRPs (already for unary queries over classical databases)

DB has row IDs if first position of each relation has globally unique value, linearly ordered

With row IDs, expressiveness gap between frontier-guarded and monadic vanishes

Theorem

Frontier-guarded NRPs = monadic NRPs
over databases with row IDs

Inherited: connection to DHNs and UQAFO



Unrestricted NRPs

FOCQ: FO with **real-valued counting terms**, interpreted on real-weighted FO structures

constants from $\mathbb{Q}$, $+$, $\cdot$, $\max$, $>$, $\sum$ aggregation terms

Variation of recent **query language FO(SUM)** for neural networks [GroheEtAl 2026]

Under suitable representation **real-weighted FO structures** $\Leftrightarrow$ **embedded databases**:

Theorem

$\text{NRP} = \text{FOCQ}$
for classical queries over embedded databases

We also use this to show:

Theorem

1. $\text{NRP} \subseteq \text{TC}^0$ for classical queries
2. $\text{NRP} = \text{TC}^0$ for classical queries over **ordered** classical databases